MTD215

Examination - July Semester 2025

Application of C++ in Multimedia

Tuesday, 11 November 2025 01:00 pm - 03:00 pm

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **FOUR (4)** questions and comprises **SIX (6)** printed pages (including cover page).
2. You must answer **ALL** questions.
3. All answers must be written in the answer book.
4. This is **Open-book** examination.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

MTD215 Copyright © 2025 Singapore University of Social Sciences (SUSS)

Answer all questions. (Full marks: 100)

Question 1

(a) You are required to implement a C++ class to simulate the motion of a graphical object in a 2D plane.

Construct a class `MovingObject` with the following attributes:

- `string name` – name of the object
- `float x, y` – current position coordinates
- `float vx, vy` – velocity components in x and y directions
- `float ax, ay` – acceleration components

Initialize them via a parameterized constructor.

(6 marks)

(b) Write a member function updatePosition(float t) that updates the object's position using the basic kinematic equations:

x = x + vx * t + 0.5 * ax * t * t

vx = vx + ax * t

Repeat similarly for y.

(6 marks)

(c) Add a `printStatus()` function that prints the name and current position of the object formatted as:

Object <name> is at position (<x>, <y>)

Describe the logic.

(4 marks)

(d) In main(), create two MovingObject instances and simulate their movement in a loop. Call updatePosition() and printStatus() for each iteration.

(5 marks)

(e) Modify the class to include a static counter for how many objects have been created. Print this value at the end of the program.

(4 marks)

Question 2

(a) Explain the **FOUR (4)** core principles of Object-Oriented Programming with relevant C++ code snippets that demonstrate each concept. Briefly appraise the use.

(12 marks)

(b) Provide pseudo-code for a typical SDL event-handling loop that handles keyboard and mouse events for a user interface.

(8 marks)

(c) SDL natively supports .bmp images only. Outline the **FIVE (5)** steps required to extend an SDL program to support .png image rendering using SDL_image.

(5 marks)

Question 3

Refer to the following class and provide detailed code comments:

```
class Shape {
public:
   Shape() { width = height = 10; }
   Shape(int w, int h) : width(w), height(h) {}
   ~Shape() {}
   Shape operator*(const Shape& s) {
      return Shape(width * s.width, height * s.height);
   }
   int getArea() { return width * height; }
private:
   int width;
   int height;
};
int main() {
   Shape A;
   Shape B(4, 5);
   Shape C = A * B;
   cout << "Area of Shape C: " << C.getArea() << endl;
   return 0;
}
```

(a) Describe line-by-line comments for all functions, constructors, operator overloading, and object instantiation in the above code.

(10 marks)

(b) Modify the class to include a static variable to count the number of objects created, and update the constructor to increment it. Implement a function to display this count.

(6 marks)

(c) Write a destructor message to confirm when objects are destroyed and explain object lifetime in `main()`.

(4 marks)

(d) Explain how the use of operator overloading helps in multimedia simulations (e.g., in vector or pixel operations). Identify an example.

(5 marks)

MTD215 Copyright © 2025 Singapore University of Social Sciences (SUSS)

Question 4

You are provided with the following partial SDL program:

```
#include <SDL.h>
#include <iostream>
using namespace std;
const int WIDTH = 640;
const int HEIGHT = 480;
const int GAP = 50;
const int COLOR_SPEED = 2;
void get_rectangle_data(int* w, int* h) {
   cout << "Enter width: ";
   cin >> *w;
   cout << "Enter height: ";
   cin >> *h;
}
void draw_outer_rect(SDL_Renderer* renderer, SDL_Rect* rect) {
   SDL_SetRenderDrawColor(renderer, 0, 255, 0, 255);
   SDL_RenderFillRect(renderer, rect);
}
void draw_inner_rect(SDL_Renderer* renderer, SDL_Rect* rect, int r, int g, int b) {
   SDL_SetRenderDrawColor(renderer, r, g, b, 255);
   SDL_RenderFillRect(renderer, rect);
}
void cycle_rgb_color(int* r, int* g, int* b) {
   *r = (*r + COLOR_SPEED) % 256;
   *g = (*g + COLOR_SPEED) % 256;
   *b = (*b + COLOR_SPEED) % 256;
}
```

(a) Complete the `main()` function that:

- Initializes SDL and creates a window
- Uses the above helper functions
- Continuously updates the inner rectangle's color and redraws the screen

Quits SDL on window close event. Explain how it works briefly.

(10 marks)

(b) Identify and describe **FIVE (5)** bugs or missing elements in the above SDL code and suggest fixes. Examples: forgetting `SDL_RenderPresent`, missing `SDL_Delay`, undeclared variables, etc.

(5 marks)

(c) Judge and explain the expected visual behavior of the program. What happens to the rectangles, colors, and interaction over time?

(5 marks)

(d) Suggest **TWO (2)** modifications to the design that would enhance interactivity or aesthetics (e.g., mouse interaction, color transitions, border effects).

(5 marks)

----- END OF PAPER -----